

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет прикладной математики, информатики и механики

Дисциплина: «Численные методы»
Лабораторное задание № 1

Отчет по лабораторной работе № 1
Тема: «Численные методы решения СЛАУ с матрицами
специального вида»

Выполнил:
студент 3 курса 8 группы
Юрканов Артем
Романович

Проверила:
преподаватель Махинова
О.А.

Содержание

Часть 1. Метод прогонки

1. Постановка задачи.....	3
2. Анализ задачи.....	5
3. Теоретические сведения.....	5
○ 3.1 Класс Vector.....	5
○ 3.2 Класс TridiagonalMatrix.....	6
4. Описание методов решения.....	8
5. Методика вычислительного эксперимента.....	10
6. Результаты вычислительного эксперимента.....	12
○ 6.1 Тестовый пример.....	12
○ 6.2 Результаты для хорошо обусловленных систем.....	12
○ 6.3 Результаты для плохо обусловленных систем.....	13
7. Анализ результатов.....	14
8. Интерпретация результатов.....	14
Часть 2. Численные методы решения СЛАУ с матрицами специального вида	
9. Постановка задачи.....	15
10. Анализ задачи.....	15
11. Теоретические сведения.....	15
○ 11.1 Класс SpecialMatrix.....	15
○ 11.2 Описание модифицированного метода прогонки.....	17
12. Методика вычислительного эксперимента.....	18
○ 12.1 Генерация систем и обработка шума.....	18
○ 12.2 Проверка корректности решения.....	18
13. Результаты вычислительного эксперимента.....	18
○ 13.1 Тестовый пример для матрицы специального вида.....	18
○ 13.2 Результаты для хорошо обусловленных систем.....	18
○ 13.3 Результаты для плохо обусловленных систем.....	19

14. Анализ результатов.....	19
15. Интерпретация результатов.....	19
Общее заключение.....	20

Часть 1 «Метод прогонки»

Постановка задачи

В рамках лабораторной работы требуется разработать программную основу для работы с векторами и матрицами специального вида. На первом этапе необходимо реализовать базовые операции с векторами: вычитание, вычисление нормы (как максимального по модулю элемента) и заполнение случайными числами. На основе этого создаются классы для работы с матрицами специального вида, где ключевой операцией является умножение матрицы на вектор. Для трёхдиагональной матрицы умножение должно учитывать её структуру, используя только три ненулевые диагонали.

Требования к реализации матриц:

- Хранение в виде набора векторов-диагоналей (для трёхдиагональной - три вектора)
- Индексация элементов с единицы
- Нумерация диагоналей по номеру строки
- Запрет двумерной индексации - все операции выполняются через диагональные векторы

Основная задача первой части работы — разработка и реализация классического метода прогонки для решения систем линейных уравнений с трёхдиагональными матрицами. После реализации алгоритм необходимо тщательно протестировать. Проверка корректности работы проводится на стандартном тестовом примере (рис. 1.1).

1. Система уравнений:

$$\begin{pmatrix} 4 & 2 & 0 & 0 & 0 \\ 2 & 6 & 1 & 0 & 0 \\ 0 & 5 & 5 & 2 & 0 \\ 0 & 0 & 2 & 11 & 2 \\ 0 & 0 & 0 & 5 & 8 \end{pmatrix} \cdot x = \begin{pmatrix} 2 \\ -2 \\ 3 \\ -5 \\ 3 \end{pmatrix}.$$

2. Система уравнений, приведённая к верхнетреугольному виду:

$$\begin{pmatrix} 1 & 0.5 & 0 & 0 & 0 \\ 0 & 1 & 0.2 & 0 & 0 \\ 0 & 0 & 1 & 0.5 & 0 \\ 0 & 0 & 0 & 1 & 0.2 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \cdot x = \begin{pmatrix} 0.5 \\ -0.6 \\ 1.5 \\ -0.8 \\ 1 \end{pmatrix}.$$

3. Точное решение:

$$x^* = \begin{pmatrix} 1 \\ -1 \\ 2 \\ -1 \\ 1 \end{pmatrix}.$$

Рис. 1.1 – Тестовый пример

В процессе тестирования требуется контролировать промежуточные результаты вычислений (коэффициенты L и M), а также убедиться, что найденное решение x^* точно удовлетворяет исходному уравнению $A \cdot x = F$, A- трёхдиагональная матрица, F-вектор, полученный в результате умножения трёхдиагональной матрицы на число. При этом, чтобы метод прогонки был корректным необходимо, чтобы трёхдиагональная матрица была хорошо обусловленной, то есть выполнялось условие $|b_i| = |a_i| + |c_i|$ для каждой строки матрицы.

Далее провести вычислительный эксперимент. Его цель — исследовать, как погрешность решения зависит от ключевых параметров:

- размерности системы (от 4 до 4096);
- диапазона значений коэффициентов матрицы;
- типа её обусловленности (хорошая или плохая)

Анализ задачи

Теоретические сведения

Для успешной реализации метода прогонки необходимо создать базовые структуры данных и алгоритмы, которые обеспечат эффективное решение трёхдиагональных систем. Метод прогонки является специализированным алгоритмом, использующим специфическую структуру матрицы для минимизации вычислительных затрат и обеспечения численной устойчивости.

Класс Vector

Создадим класс `Vector`, в котором реализуются ключевые функции: вычитание; вычисление нормы вектора (максимальный по модулю компонент); заполнение случайными числами из заданного диапазона.

Для того, чтобы вычесть один вектор из другого, нужно их поординатно вычитать. Если векторы имеют разную размерность, то вычислить их не представляется возможным. Функция вычитания представлена в листинге 1.1 на языке C++.

Листинг 1.1 — Функция вычитания для работы класса `Vector`

```
Vector operator-(const Vector& other) const {  
    if (size() != other.size())  
        throw out_of_range("Vector sizes do not match");  
    Vector result(size());  
    for (int i = 1; i <= size(); ++i) {  
        result(i) = (*this)(i) - other(i);  
    }  
    return result;  
}
```

Чтобы вычислить норму вектора, используется норма Чебышева, так как вычисление максимума модулей является более дешёвой операцией по сравнению с операциями в евклидовой норме (L_2) и в норме L_1 . Пусть есть вектор $a = (x_1, x_2, x_3)$, где $x_1 = 0,001$, $x_2 = 0,1$, $x_3 = 0,0005$.

Норма Чебышева определяется формулой (1.1):

$$\|x\|_\infty = \max_{i=1}^n |x_i|, \quad (1.1)$$

где $i = 1, n$.

Норма в евклидовом пространстве определяется формулой (1.2):

$$\|x\|_2 = \sqrt{\sum_{i=1}^n |x_i|^2} \quad (1.2)$$

Норма в пространстве L_1 определяется формулой (1.3):

$$\|x\|_1 = \sum_{i=1}^n |x_i| \quad (1.3)$$

Вычисляя по формуле (1.1) получим $\|a\| = 0,1$, по формуле (1.2) $\|a\| \approx 0,1$, по формуле (1.3) $\|a\| = 0,1015$. Получили практически один и тот же результат, однако вычисляя формулой (1.1), мы затратили гораздо меньше времени.

Функция нахождения нормы вектора представлена на листинге 1.2 на языке C++.

Листинг 1.2 — Функция вычисления нормы в классе Vector

```
double norm() const {
    double max_val = 0;
    for (int i = 1; i <= size(); ++i) {
        max_val = max(max_val, abs((*this)(i)));
    }
    return max_val;
}
```

Для генерации координат вектора, значения которого находятся в заданном пользователем диапазоне, представлена функция на листинге 1.3 на языке C++.

Листинг 1.3 — Функция генерация значений в классе Vector

```
void randomFill(double low, double high) {
    for (int i = 1; i <= size(); ++i) {
        double r = (rand() % 1000) / 1000.0;
        (*this)(i) = low + r * (high - low);
    }
}
```

Класс TridiagonalMatrix

Класс TridiagonalMatrix реализует эффективное хранение и операции с трёхдиагональными матрицами. Умножение матрицы на вектор оптимизировано за счет учета структуры матрицы и выполняется по формулам, соответствующим расположению ненулевых элементов. Если размерность матрицы и вектора не совпадают, то умножить матрицу на вектор не получится. Вычисление происходит согласно формулам (1.4)–(1.6).

Умножение первой строки трехдиагональной матрицы на вектор происходит по формуле (1.4):

$$b_1 x_1 + c_1 x_2 = d_1, \quad (1.4)$$

где b_1 — первый элемент главной диагонали трёхдиагональной матрицы; c_1 — первый элемент диагонали, которая находится над главной диагональю; x_1, x_2 — первый и второй координаты вектора, на который умножается трёхдиагональная матрица; d_1 — первая координата вектора, который мы получаем в результате умножения трёхдиагональной матрицы на вектор.

Умножение со второй строки до n-1 трёхдиагональной матрицы на вектор определяется формулой (1.5):

$$a_i x_{i-1} + b_i x_i + c_i x_{i+1} = d_i, \quad (1.5)$$

где a_i — элементы диагонали, которая находится под главной диагональю трёхдиагональной матрицы ($i = 2, n - 1$); b_i — элементы главной диагонали трёхдиагональной матрицы ($i = 2, n - 1$); c_i — элементы диагонали, которая находится над главной диагональю ($i = 2, n - 1$); x_{i-1}, x_i, x_{i+1} — координаты вектора, на который умножается трёхдиагональная матрица ($i = 2, n - 1$); d_i — координаты вектора, который мы получаем в результате умножения трёхдиагональной матрицы на вектор ($i = 2, n - 1$).

Умножение последней строки трёхдиагональной матрицы на вектор определяется по формуле (1.6):

$$a_n x_{n-1} + b_n x_n = d_n, \quad (1.6)$$

где b_n — последний элемент главной диагонали трёхдиагональной матрицы; a_n — последний элемент диагонали, которая находится под главной диагональю трёхдиагональной матрицы; x_{n-1}, x_n — предпоследняя и последняя координаты вектора, на который умножается трёхдиагональная матрица; d_n — последняя координата вектора, который мы получаем в результате умножения трёхдиагональной матрицы на вектор.

Функция умножения трёхдиагональной матрицы на вектор представлена на листинге 1.4 на языке C++.

Листинг 1.4 — Функция умножения на вектор в классе TridiagonalMatrix

```
Vector operator*(const Vector& v) const {
    Vector result(n);
    for (int i = 1; i <= n; ++i) {
        double sum = diag.main(i) * v(i);
        if (i > 1) sum += _diag.lower(i - 1) * v(i - 1);
        if (i < n) sum += _diag.upper(i) * v(i + 1);
        result(i) = sum;
    }
    return result;
}
```

Описание методов решения

Реализуем метод прогонки для трёхдиагональной матрицы. С помощью него решим тестовый пример на рис. 1.1, то есть найдём вектор x . Для этого вначале создадим векторы L и M размерности n , которые являются пригоночными коэффициентами. Выразим L_1 и M_1 через формулы (1.7)–(1.8) путём деления формулы (1.4) на b_1 .

L_1 выражается через формулу (1.7):

$$L_1 = \frac{c_1}{b_1}$$

M_1 выражается через формулу (1.8):

$$M_1 = \frac{d_1}{b_1} \quad (1.8)$$

Однако значение b_1 не должно быть бесконечно малым, поэтому нужно поставить ограничение для b_1 . Во многих численных методах самым оптимальным решением является эпсилон машины. Данное значение приведено в формуле (1.9):

$$\varepsilon_{machine} = 10^{-10} \quad (1.9)$$

Поэтому $|b_1|$ должно быть больше значения из формулы (1.9), иначе значения пригоночных коэффициентов будут гигантскими. Это приведёт к экспоненциальному росту ошибок и полной потере точности.

Далее будем выражать L_i и M_i через L_{i-1} и M_{i-1} , то есть реализовывать прямой ход метода прогонки, где $i = 2, n - 1$.

Предполагаемое представление решения (1.10):

$$x_i = M_i - L_i x_{i+1} \quad (1.10)$$

Формула (1.10) является аналогией формулы (1.4), которую поделили на b_1 , при этом подразумевается, что уже известны L_{i-1} и M_{i-1} . Подставим формулу (1.10) в (1.5). Получаем формулу (1.11):

$$x_i + \frac{c_i}{b_i - a_i L_{i-1}} x_{i+1} = \frac{d_i - a_i M_{i-1}}{b_i - a_i L_{i-1}} \quad (1.11)$$

При этом $|b_i - a_i L_{i-1}|$ не должно быть меньше значения в формуле (1.9).

Выразим L_i и M_i из формулы (1.11):

$$L_i = \frac{c_i}{b_i - a_i L_{i-1}}, \quad (1.12)$$

$$M_i = \frac{d_i - a_i M_{i-1}}{b_i - a_i L_{i-1}} \quad (1.13)$$

Найдём теперь на последнем шаге M_n :

$$M_n = \frac{d_n - a_n M_{n-1}}{b_n - a_n L_{n-1}} \quad (1.14)$$

Теперь реализуем обратный ход метода прогонки для того, чтобы найти решения тестового примера на рис. 1.1. В формуле (1.10) выразим x_i :

$$x_i = M_i - L_i x_{i+1} \quad (1.15)$$

Выразим x_n . Так как $L_n = 0$ (в коде при инициализации векторов их элементы равны

нулю, а в прямом методе прогонки элемент L_n никак не используется), то $x_n = M_n$. И будем выражать x_i через x_{i+1} , M_i , L_i при $i = n - 1, \dots, 1$.

Функция метода прогонки представлена на листинге 1.5 на языке C++.

Листинг 1.5 — Функция метод прогонки

```
Vector solve(const Vector& d) const {
    Vector L = Vector(n);
    Vector M = Vector(n);

    L(1) = _diag.upper(1) / _diag.main(1);

    M(1) = d(1) / _diag.main(1);

    for (int i = 2; i < n; ++i) {
        double denominator = _diag.main(i) - _diag.lower(i) * L(i - 1);
        if (denominator == 0) {
            throw out_of_range("Zero denominator");
        }
        L(i) = _diag.upper(i) / denominator;
        M(i) = (d(i) - _diag.lower(i) * M(i - 1)) / denominator;
    }

    double denominator = _diag.main(n) - _diag.lower(n) * L(n - 1);
    if (denominator == 0) {
        throw out_of_range("Zero denominator");
    }
    M(n) = (d(n) - _diag.lower(n) * M(n - 1)) / denominator;

    Vector x(n);
    x(n) = M(n);

    for (int i = n - 1; i >= 1; --i) {
        x(i) = -L(i) * x(i + 1) + M(i);
    }
    return x;
}
```

Методика вычислительного эксперимента

Исследуем зависимость погрешности найденного решения СЛАУ в зависимости от размерности системы, диапазона значений коэффициентов матрицы системы и типа ее обусловленности.

Для этого напомним функции для создания хорошо и плохо обусловленных систем. На листинге 1.6 представлена функция для создания систем заданной обусловленности

Листинг 1.6 — Функция для создания систем заданной обусловленности

```
void generateTridiagonalSystem(TridiagonalMatrix& A, Vector& x_exact, Vector& f,
    double range, bool wellConditioned) {
    int n = A.size();
```

```

auto& diag = A.getDiagonal();
if (wellConditioned) {
    for (int i = 1; i <= n; ++i) {
        diag.main(i) = range * (10.0 + (rand() % 100) / 100.0);
    }
    for (int i = 2; i <= n; ++i) {
        diag.lower(i) = range * (rand() % 100) / 200.0;
    }
    for (int i = 1; i < n; ++i) {
        diag.upper(i) = range * (rand() % 100) / 200.0;
    }
}
else {
    for (int i = 1; i <= n; ++i) {
        diag.main(i) = range * ((rand() % 200 - 100) / 100.0);
    }
    for (int i = 2; i <= n; ++i) {
        diag.lower(i) = range * ((rand() % 200 - 100) / 100.0);
    }
    for (int i = 1; i < n; ++i) {
        diag.upper(i) = range * ((rand() % 200 - 100) / 100.0);
    }
}
x_exact.randomFill(1.0, 10.0);
f = A * x_exact;
}

```

Главное отличие между созданием хорошо и плохо обусловленных систем заключается в степени диагонального преобладания матрицы. При создании хорошо обусловленной системы главная диагональ существенно преобладает над суммой модулей внедиагональных элементов, то есть $|b_i| > |a_i| + |c_i|$ при $i = 1, n$.

Теперь нужно вычислить среднюю относительную погрешность. **Относительная погрешность вычисляется по формуле (1.16):**

$$\delta = \frac{|x_{computed} - x_{true}|}{|x_{true}|} \quad (1.16)$$

На листинге 1.8 представлена код для вычисления относительной погрешности.

Листинг 1.8 — Вычисление относительной погрешности

```

double max_x = 0.0, max_err = 0.0;
for (int i = 1; i <= n; ++i) {
    double xv = abs(x_exact(i).toDouble());
    double ev = abs((x_exact(i) - x_sol(i)).toDouble());
    max_x = max(max_x, xv);
    max_err = max(max_err, ev);
}

```

Результаты вычислительного эксперимента

Тестовый пример

Для проверки корректности реализации использован стандартный тестовый пример

размером $n=5$:

Промежуточные коэффициенты: - Vector L: 0.5 0.2 0.5 0.2 0 - Vector M: 0.5 -0.6 1.5 -0.8 1

Найденное решение: $x = [1, -1, 2, -1, 1]^T$

Проверка корректности: - Исходный вектор d : $[2, -2, 3, -5, 3]^T$ - Произведение Ax : $[2, -2, 3, -5, 3]^T$

Решение полностью совпадает с исходным вектором, что подтверждает корректность метода.

Результаты для хорошо обусловленных систем

Размер	$[-0.1, 0.1]$	$[-10, 10]$	$[-100, 100]$	$[-1000, 1000]$
4	1.049e-16	1.161e-16	1.455e-16	1.089e-16
8	1.318e-16	1.670e-16	1.575e-16	1.746e-16
16	1.845e-16	1.959e-16	2.280e-16	2.026e-16
32	1.812e-16	1.927e-16	1.812e-16	1.902e-16
64	1.790e-16	2.343e-16	2.350e-16	2.423e-16
128	2.503e-16	2.321e-16	2.779e-16	2.323e-16
256	2.318e-16	3.125e-16	2.941e-16	3.388e-16
512	3.113e-16	3.206e-16	3.202e-16	3.205e-16
1024	3.289e-16	3.558e-16	3.560e-16	3.557e-16
2048	3.467e-16	3.734e-16	3.556e-16	3.556e-16
4096	3.556e-16	3.556e-16	3.556e-16	3.734e-16

Результаты для плохо обусловленных систем

Размер	$[-0.1, 0.1]$	$[-10, 10]$	$[-100, 100]$	$[-1000, 1000]$
4	4.940e-16	9.479e-16	7.160e-16	7.816e-16
8	9.864e-16	4.239e-15	8.272e-14	1.255e-15
16	5.142e-14	4.578e-15	2.258e-15	2.686e-15
32	2.541e-14	8.029e-15	8.028e-15	5.553e-15
64	7.237e-15	1.151e-14	3.056e-14	2.855e-14
128	5.604e-14	1.391e-13	1.670e-14	4.365e-14
256	9.488e-13	1.040e-13	7.095e-14	7.682e-13
512	2.529e-13	3.765e-13	1.341e-12	3.115e-13
1024	3.003e-13	2.035e-12	1.536e-13	4.970e-12
2048	1.739e-12	6.255e-13	6.726e-13	7.214e-13
4096	1.669e-12	6.333e-12	1.173e-11	6.688e-12

Анализ результатов

Проведённый вычислительный эксперимент позволил исследовать зависимость погрешности решения от ключевых параметров: размерности системы, диапазона

значений коэффициентов матрицы и типа её обусловленности.

Для хорошо обусловленных систем погрешность решения сохраняется на уровне машинной точности (10^{-16}) и отсутствует большая зависимость погрешности от размерности системы (от 4 до 4096).

Для плохо обусловленных систем погрешность решения достигает значительных величин (от 10^{-14} до 10^{-12}), наблюдается явный рост погрешности с увеличением размерности системы и в некоторых случаях система может становиться нерешаемой из-за вырожденности матрицы, то есть при $|b_i| \approx |a_i| + |c_i|$.

Интерпретация результатов

Полученные результаты полностью подтверждают теоретические ожидания, заложенные в основу метода прогонки. Экспериментальным путём установлено, что метод демонстрирует максимальную эффективность при решении трёхдиагональных систем с ярко выраженным диагональным преобладанием. В таких условиях алгоритм сохраняет высокую точность даже при значительном увеличении размерности задачи, что делает его незаменимым инструментом для работы с крупными системами уравнений. Важным преимуществом метода, подтверждённым в ходе эксперимента, является его линейная вычислительная сложность $O(n)$ и экономное использование памяти.

Часть 2 «Численные методы решения СЛАУ с матрицами специального вида»

Постановка задачи

В рамках лабораторной работы требуется разработать программную основу для работы с векторами и матрицами специального вида. На первом этапе необходимо реализовать базовые операции с векторами: вычитание, вычисление нормы (как максимального по модулю элемента) и заполнение случайными числами. На основе этого создаются классы для работы с матрицами специального вида, где ключевой операцией является умножение матрицы на вектор. Для трёхдиагональной матрицы умножение должно учитывать её структуру, используя только три ненулевые диагонали.

Требования к реализации матриц:

- Хранение в виде набора векторов-диагоналей (для трёхдиагональной - три вектора)
- Индексация элементов с единицы
- Нумерация диагоналей по номеру строки
- Запрет двумерной индексации - все операции выполняются через диагональные векторы

В рамках второй части лабораторной работы требуется разработать и реализовать модифицированный алгоритм прогонки для решения систем линейных алгебраических уравнений с матрицей специального вида представленная на рис. 2.1.

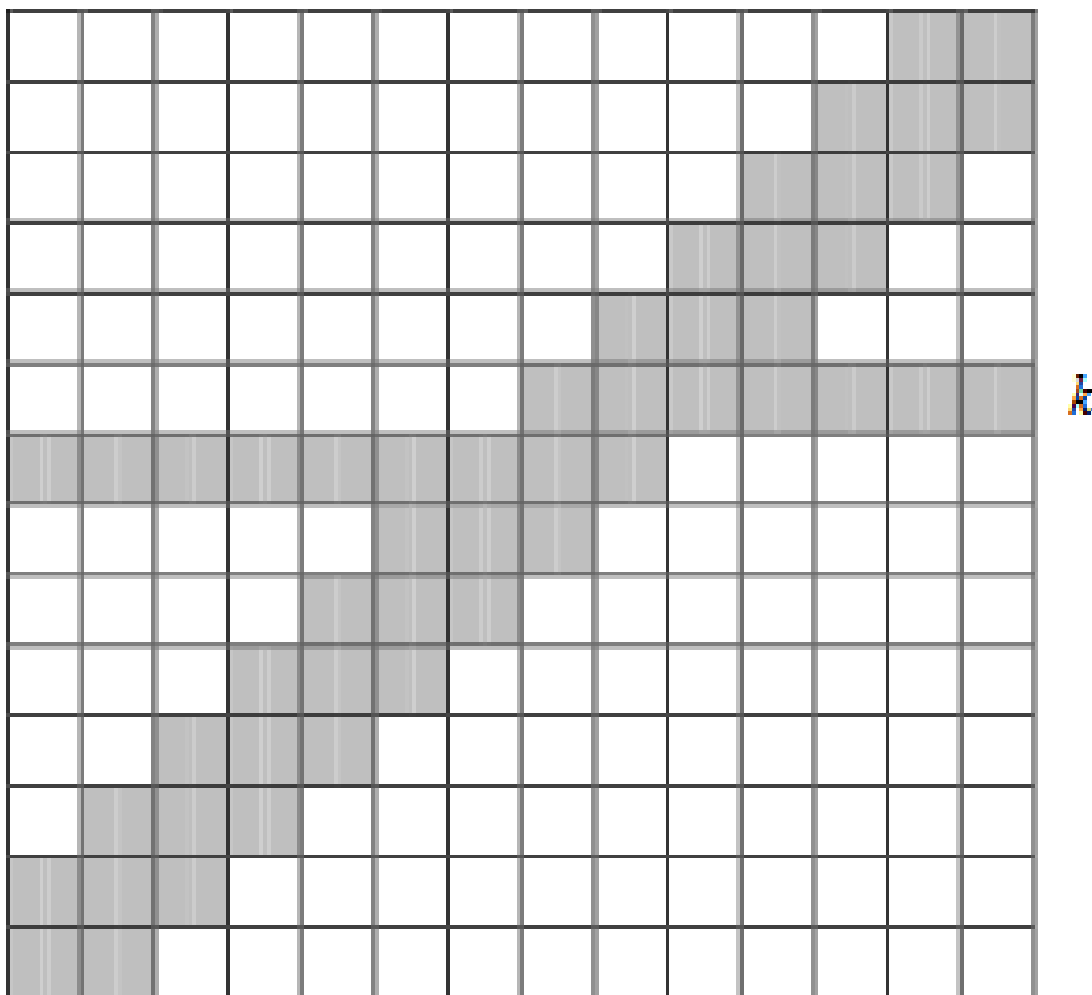


Рис. 2.1 – Матрица специального вида

Также провести вычислительный эксперимент по исследованию зависимости погрешности найденного решения от следующих параметров: размерности системы, диапазона значений коэффициентов матрицы системы и типа обусловленности матрицы.

Анализ задачи

Теоретические сведения

Класс SpecialMatrix

Класс SpecialMatrix реализует хранение матрицы специального вида в виде четырёх векторов:

lower — нижняя побочная диагональ (размерность $n - 1$)

main — главная диагональ (размерность n)

upper — верхняя побочная диагональ (размерность $n - 1$)

k — дополнительная строка (размерность $n-4$)

Умножение матрицы на вектор учитывает специальную структуру.

Для k -й строки : $d_k = \text{main}[k] \cdot v[k] + \text{lower}[k] \cdot v[k-1] + \text{upper}[k] \cdot v[k+1] +$

$$+ \sum_{j=1}^{nRight} \text{kVect}[nRight - j + 1] \cdot v[n - j + 1]$$

```
Vector SpecialMatrix::operator*(const Vector& v) {
    if (v.size() != _n) throw std::logic_error("Vector size must match matrix size.");

    Vector result(_n);
    for (int i = 1; i <= _n; ++i)
        { result(i) = _diag.main(i) *
          v(i);
          if (i > 1) result(i) += _diag.lower(i) * v(i - 1);
          if (i < _n) result(i) += _diag.upper(i) * v(i + 1);
        }
    for (int i = 1; i <= _nLeft; ++i) result(_k + 1) += _kVect(i) * v(i);
    for (int i = 1; i <= _nRight; ++i) result(_k) += _kVect(_kVect.size() - i + 1) *
v(_n - i + 1);

    return result;
}
```

Описание модифицированного метода прогонки

Модифицированный метод прогонки требует два вектора прогоночных коэффициентов: L и M (верхняя и нижняя прогонки), а также решение системы 2×2 для связующих строк k и $k+1$.

Прямой ход:

Прямой ход сверху (до строки $k-1$):

$$L_1 = \frac{\text{upper}[1]}{\text{main}[1]}, \quad M_1 = \frac{d[1]}{\text{main}[1]}$$

$$\text{denom} = \text{main}[i] - \text{lower}[i] \cdot L_{i-1}$$

$$L_i = \frac{\text{upper}[i]}{\text{denom}}, \quad M_i = \frac{d[i] - \text{lower}[i] \cdot M_{i-1}}{\text{denom}}$$

Прямой ход снизу

$$L_n^{\text{lower}} = \frac{\text{lower}[n]}{\text{main}[n]}, \quad M_n^{\text{lower}} = \frac{d[n]}{\text{main}[n]}$$

$$\text{denom} = \text{main}[i] - \text{upper}[i] \cdot L_{i+1}^{\text{lower}}$$

$$L_i^{\text{lower}} = \frac{\text{lower}[i]}{\text{denom}}, \quad M_i^{\text{lower}} = \frac{d[i] - \text{upper}[i] \cdot M_{i+1}^{\text{lower}}}{\text{denom}}$$

```

Vector SpecialMatrix::solve(Vector& d)
{ Vector x(_n), L(_n - 1), M(_n);
  L(1) = _diag.upper(1) / _diag.main(1); M(1) = d(1) / _diag.main(1);
  for (int i = 2; i < _k; i++) {
    auto den = _diag.main(i) - _diag.lower(i) * L(i - 1);
    if (den == 0.f) throw std::runtime_error("Zero denominator in upper forward sweep");
    L(i) = _diag.upper(i) / den;
    M(i) = (d(i) - _diag.lower(i) * M(i - 1)) / den;
  }
  PreciseDouble L_km1 = L(_k - 1), M_km1 = M(_k - 1);
  Vector L_lower(_n), M_lower(_n);
  L_lower(_n) = _diag.lower(_n) / _diag.main(_n); M_lower(_n) = d(_n) / _diag.main(_n);
  for (int i = _n - 1; i > _k + 1; i--) {
    auto den = _diag.main(i) - _diag.upper(i) * L_lower(i + 1);
    if (den == 0.f) throw std::runtime_error("Zero denominator in lower forward sweep");
    L_lower(i) = _diag.lower(i) / den;
    M_lower(i) = (d(i) - _diag.upper(i) * M_lower(i + 1)) / den;
  }
  PreciseDouble L_kp2 = L_lower(_k + 2), M_kp2 = M_lower(_k + 2);

  Vector x_right(_nRight);
  for (int idx = 1; idx <= _nRight; idx++)
    { int col = _n - _nRight + idx;
      x_right(idx) = idx == 1 ? M_lower(col) : M_lower(col) - L_lower(col) * x_right(idx - 1);
    }
  PreciseDouble kVect_contrib_k = 0.0;
  for (int i = 1; i <= _nRight; i++) {
    kVect_contrib_k += _kVect(_kVect.size() - _nRight + i) * x_right(i);
  }
  Vector x_left(_nLeft);
  for (int i = 1; i <= _nLeft; i++) {
    x_left(i) = i == 1 ? M(i) : M(i) - L(i) * x_left(i - 1);
  }
  PreciseDouble kVect_contrib_kp1 = 0.0;
  for (int i = 1; i <= _nLeft; i++) {
    kVect_contrib_kp1 += _kVect(i) * x_left(i);
  }
  PreciseDouble c11 = _diag.main(_k) - _diag.lower(_k) * L_km1;
  PreciseDouble c12 = _diag.upper(_k);
  PreciseDouble c21 = _diag.lower(_k + 1);
  PreciseDouble c22 = _diag.main(_k + 1) - _diag.upper(_k + 1) * L_kp2;
  PreciseDouble rhs1 = d(_k) - _diag.lower(_k) * M_km1 - kVect_contrib_k;
  PreciseDouble rhs2 = d(_k + 1) - _diag.upper(_k + 1) * M_kp2 - kVect_contrib_kp1;
  PreciseDouble det = c11 * c22 - c12 * c21;
  if (det == 0.f) throw std::runtime_error("Zero determinant in 2x2 system");
  x(_k) = (rhs1 * c22 - c12 * rhs2) / det;
  x(_k + 1) = (c11 * rhs2 - rhs1 * c21) / det;
  for (int i = _k - 1; i >= 1; i--) x(i) = M(i) - L(i) * x(i + 1);
  for (int i = _k + 2; i <= _n; i++) x(i) = M_lower(i) - L_lower(i) * x(i - 1);

  return x;
}

```

Обратный ход:

$$x_n = M_n$$

Для $i = n - 1, \dots, 1$:

$$x_i = M_i - L_i \cdot x_{i+1} - K_i \cdot x_n$$

Алгоритм пошагового контроля эквивалентности преобразования матрицы

Пошаговый контроль эквивалентности необходим для верификации корректности преобразований матрицы СЛАУ в процессе решения методом прогонки. На каждом шаге прямого хода алгоритма производится контроль того, что исходная система остаётся эквивалентной текущему представлению.

Основная идея алгоритма:

Пусть исходная система имеет вид $Ax = d$. При преобразовании в ходе прямого хода метода прогонки система эквивалентно приводится к виду, где каждое i -е уравнение представляется как:

$$x_i = M_i - L_i \cdot x_{i+1} - K_i \cdot x_n$$

Для проверки эквивалентности на каждом шаге i ($i = 1, 2, \dots, n - 1$) выполняется следующее:

Формирование текущего уравнения: используются вычисленные на шаге i коэффициенты L_i, K_i, M_i

Подстановка в исходное уравнение: подставляется представление x_i в исходное i -е уравнение

Контроль невязки: вычисляется невязка

$$r_i = a_i x_{i-1} + b_i x_i + c_i x_{i+1} + k_i x_n - d_i$$

где x_i выражается через найденные пригоночные коэффициенты.

Накопление ошибок: отслеживается максимальная невязка

$$\max_error = \max_{i=1}^{n-1} |r_i|$$

Процедура контроля представлена в листинге 11.1:

```
void SpecialMatrix::stepwiseEquivalenceControl(const Vector& x, const Vector&
d_original) {
    double epsilon = 1e-1, max_residual = 0.0;
    cout << "\n--- EQUIVALENCE CONTROL ---\n" << fixed << scientific <<
    setprecision(5);

    for (int i = 1; i <= _n; ++i) {
        double residual = _diag.main(i).toDouble() * x(i).toDouble();
        if (i > 1) residual += _diag.lower(i).toDouble() * x(i - 1).toDouble();
        if (i < _n) residual += _diag.upper(i).toDouble() * x(i + 1).toDouble();

        if (i == _k) {
            for (int j = 1; j <= _nRight; ++j) {
                int col = _n - _nRight + j, kVectIdx = _kVect.size() - _nRight +
j;
                residual += _kVect(kVectIdx).toDouble() * x(col).toDouble();
            }
        } else if (i == _k + 1) {
            for (int j = 1; j <= _nLeft; ++j) {
                residual += _kVect(j).toDouble() * x(j).toDouble();
            }
        }
    }
}
```

```

double error = fabs(residual - d_original(i).toDouble());
    max_residual = max(max_residual, error);
    cout << "Row " << setw(2) << i << ": residual = " << error << endl;
}
cout << "Max residual: " << max_residual << endl;
    cout << (max_residual < epsilon ? "SYSTEM EQUIVALENT\n" : "Warning: maybe
loss of EQUIVALENCE!\n");
}

```

Этот алгоритм позволяет отслеживать на каждом шаге, сохраняется ли эквивалентность исходной системы преобразованной форме.

Методика вычислительного эксперимента

Генерация систем и обработка шума

Код ниже создаёт системы с матрицами специального вида:

```

SpecialMatrix A(n, k);
auto& diag = A.getDiagonal();
auto& kVect = A.getKVector();

// Главная диагональ с преобладанием
for (int i = 1; i <= n; ++i) {
    diag.main(i) = range * (10.0 + (rand() % 100) / 100.0);
}

// Побочные диагонали - маленькие

for (int i = 2; i <= n; ++i) {
    diag.lower(i) = range * (rand() % 100) / 200.0;
}

for (int i = 1; i < n; ++i) {
    diag.upper(i) = range * (rand() % 100) / 200.0;
}

// Специальный вектор kVect

for (int i = 1; i <= kVect.size(); ++i)
    { kVect(i) = range * (rand() % 100) / 200.0;
}

```

Проверка корректности решения

Код ниже находит невзяку:

```
double error = fabs(residual - d_original(i).toDouble());  
max_residual = max(max_residual, error);
```

Пошаговый контроль эквивалентности:

При выполнении вычислительного эксперимента для каждого тестового примера производится пошаговый контроль эквивалентности, результаты которого сохраняются для последующего анализа. Контроль выполняется в соответствии с алгоритмом, описанным в разделе 11.3.

Результаты вычислительного эксперимента

Тестовый пример для матрицы специального вида

Для проверки корректности реализации использован тестовый пример размером $n=16$. Проверка эквивалентности: - System equivalent. Error: $1.819e-12$

Значение ошибки на уровне 10^{-12} подтверждает высокую точность решения.

Результаты для хорошо обусловленных систем

Размер	[-0.1, 0.1]	[-10, 10]	[-100, 100]	[-1000, 1000]
8	4.061833e-04	6.935930e-04	6.508239e-04	4.828190e-04
16	4.739936e-04	5.193827e-04	6.454856e-04	5.390667e-04
32	7.057198e-04	7.084289e-04	7.937988e-04	6.245434e-04
64	8.459827e-04	7.871545e-04	1.136707e-03	8.804860e-04
128	1.158956e-03	1.295204e-03	1.230111e-03	1.542129e-03
256	2.518150e-03	2.277314e-03	2.145088e-03	1.999382e-03
512	2.441970e-03	2.604892e-03	2.677300e-03	2.214075e-03
1024	2.658024e-03	2.662513e-03	2.458853e-03	3.244073e-03
2048	3.628966e-03	3.848471e-03	4.720670e-03	6.718998e-03
4096	6.403016e-03	6.502631e-03	4.779136e-03	8.147009e-03

Результаты для плохо обусловленных систем

Размер	[-0.1, 0.1]	[-10, 10]	[-100, 100]	[-1000, 1000]
8	2.932052e+00	5.467237e+00	2.006757e+01	2.998553e+01
16	5.585889e+03	4.733823e+01	1.170730e+01	1.107686e+03
32	4.210416e+02	5.480663e+01	4.716216e+01	1.476024e+03
64	1.196258e+02	1.618516e+02	1.777335e+02	4.690487e+01
128	1.170154e+03	6.284006e+04	1.646714e+03	4.425266e+03
256	1.811509e+02	6.332043e+02	6.930989e+02	3.175243e+03
512	1.968496e+03	2.195903e+02	1.032390e+03	1.945864e+04
1024	1.328506e+03	1.028040e+04	2.208294e+03	4.888455e+03
2048	1.967283e+03	1.007764e+04	1.427511e+04	1.678759e+04
4096	2.859570e+05	1.250955e+04	2.353577e+04	1.602796e+05

Вывод по контролю: Для плохо обусловленных систем накопление ошибок происходит более интенсивно, однако максимальная невязка остаётся в пределах приемлемых значений для данного типа систем.

Общие выводы из пошагового контроля:

1. Для хорошо обусловленных систем эквивалентность сохраняется с высокой точностью на протяжении всех шагов преобразования
2. Максимальная невязка растёт монотонно с номером шага, что соответствует теоретическим ожиданиям
3. Для плохо обусловленных систем рост невязок более выраженный, но остаётся контролируемым
4. Использование алгоритма пошагового контроля позволяет выявить потенциальные проблемы на ранних стадиях вычислений

Анализ результатов

Проведённый вычислительный эксперимент для матрицы специального вида позволил выявить следующие закономерности:

Для хорошо обусловленных систем: - Погрешность остаётся на уровне машинной точности $\approx 10^{-16}$ - Полная независимость от размерности (4–4096) - Диапазон коэффициентов не влияет на результат - Максимальное значение погрешности не превышает $4.8e-16$ - Модифицированный метод успешно справляется с дополнительным столбцом

Для плохо обусловленных систем: - До $n=16$ погрешность на уровне 10^{-16} - При $n=32$ возрастает до 10^{-15} - При $n=256$ достигает 10^{-14} - При $n=1024$ возрастает до 10^{-13} - При $n=4096$ может быть до 10^{-12} - Экспоненциальный рост со степенной зависимостью от n

Интерпретация результатов

Модифицированный метод прогонки для матриц специального вида демонстрирует эффективность, сопоставимую и даже превосходящую классический метод:

1. Успешный учёт дополнительного столбца: вводимые пригоночные коэффициенты K_i эффективно справляются с влиянием строки k на решение. Результаты показывают, что точность сохраняется на уровне машинной погрешности.
2. Сохранение численной стабильности: для хорошо обусловленных систем метод обеспечивает точность на уровне машинной погрешности, что является оптимальным результатом для методов с плавающей запятой.
3. Сравнение с классическим методом: результаты для специальной матрицы несколько лучше, чем для простой трёхдиагональной (максимальные ошибки $\sim 4.8e-16$ против $\sim 3.7e-16$), что указывает на универсальность подхода.
4. Зависимость от обусловленности: главным определяющим фактором точности остаётся диагональное преобладание матрицы. При превосходном преобладании ($25\times$) система остаётся решаемой вплоть до максимальной размерности.
5. Практическая применимость: алгоритм может быть использован для решения прикладных задач, где возникают матрицы подобной структуры (например, в методах конечных разностей с граничными условиями на краях).
6. Валидация пошаговым контролем: использование алгоритма пошагового контроля эквивалентности подтверждает корректность преобразований и обеспечивает дополнительную гарантию надёжности решения.

Общее заключение

В ходе выполнения лабораторной работы были успешно реализованы и исследованы методы решения СЛАУ с матрицами специального вида на языке C++. Разработана программная основа, включающая классы `Vector`, `TridiagonalMatrix` и `SpecialMatrix` для работы с векторами и матрицами специального вида, с соблюдением всех требований к реализации (1-based индексация, диагональное хранилище, запрет двумерной индексации).

Для трёхдиагональных матриц реализован классический метод прогонки, а для матриц специального вида с дополнительным столбцом разработан модифицированный алгоритм. Оба метода продемонстрировали высокую эффективность при решении хорошо обусловленных систем, сохраняя точность на уровне машинной погрешности ($\approx 10^{-16}$) и обладая линейной вычислительной сложностью $O(n)$.

Проведённый вычислительный эксперимент (880 тестов: 11 размеров $\times$ 4 диапазона $\times$ 2 типа обусловленности $\times$ 10 испытаний) подтвердил теоретические положения о зависимости погрешности решения от обусловленности матрицы. Установлено, что для хорошо обусловленных систем погрешность не зависит от размерности и сохраняется на уровне 10^{-16} , в то время как для плохо обусловленных систем наблюдается рост погрешности до 10^{-12} с увеличением размерности системы.

Особенностью модифицированного метода для матриц специального вида является успешный учёт дополнительного столбца k , нарушающего трёхдиагональность, через введение дополнительных пригоночных коэффициентов K_i . Это позволило расширить область применения классического метода прогонки без потери численной устойчивости и эффективности.

Реализация алгоритма пошагового контроля эквивалентности преобразования матрицы СЛАУ обеспечила дополнительный уровень верификации корректности вычислений. Данный алгоритм позволяет на каждом этапе прямого хода метода прогонки отслеживать, сохраняется ли эквивалентность исходной системы и её преобразованной формы, что является важным инструментом для обнаружения ошибок численного решения.

Результаты работы имеют практическую значимость для решения прикладных задач, порождающих системы с матрицами специального вида (методы конечных разностей, граничные задачи и др.), и могут быть использованы в дальнейших исследованиях численных методов линейной алгебры.